iSMART

Классификация на реальном датасете

Практикум для 10–11 классов и СПО

подготовка признаков → обучение модели → предсказания



На этом уроке обучающиеся проходят полный путь от чтения таблицы до первых предсказаний модели.

Что должно получиться

Прочитать датасет и понять структуру таблицы.

Подготовить признаки X и целевую переменную y.

Обучить простую модель классификации.

Получить предсказания для тестовых и новых объектов.

1. Прочитать датасет

посмотреть строки, столбцы и типы данных

2. Подготовить признаки

выбрать X, y и убрать лишние строки

3. Обучить модель

использовать дерево решений

4. Получить предсказания

посмотреть, какой класс дала модель

От учебного примера к реальному датасету

Урок 3.5

признаки были заранее придуманы: длина сообщения, ссылка, скидка, срочность

Урок 3.6

данные взяты из реальной таблицы: измерения объектов и их класс

Новый акцент

перед обучением надо понять структуру таблицы и подготовить признаки

В реальной задаче модель редко получает идеально готовую таблицу. Поэтому работа начинается не с fit(), а с внимательного чтения данных.

Датасет Iris: объект, признаки, класс

В практикуме используется классический табличный датасет Iris: измерения цветков ириса и их вид.

объект
один цветок

признаки
4 измерения

класс
species

4 признака
sepal_length_cm
sepal_width_cm
petal_length_cm
petal_width_cm

3 класса
setosa
versicolor
virginica

Шаг 1. Смотрим на таблицу

```
import pandas as pd

df = pd.read_csv('iris_real_dataset.csv')

df.head()
df.shape
df.columns
df.info()
```

Сколько строк и столбцов в таблице?

Какие признаки числовые?

Где находится целевая переменная species?

Есть ли пропуски или странные типы данных?

Задача блока — не просто запустить команды, а прочитать, что таблица сообщает о себе.

Шаг 2. Готовим признаки и целевую переменную

```
df = df.dropna()

X = df[[
    'sepal_length_cm',
    'sepal_width_cm',
    'petal_length_cm',
    'petal_width_cm'
]]

y = df['species']
```

X

таблица признаков, по которым модель будет учиться

y

правильный класс для каждого объекта

dropna()

убирает строки с пропусками, если они мешают обучению

Шаг 3. Делим данные на train и test

Модель учится на одной части таблицы, а предсказания смотрим на другой. Так мы имитируем ситуацию новых объектов.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

print(X_train.shape)
print(X_test.shape)
```

train

на этой части таблицы модель учится

test

на этой части смотрим ответы на новых примерах

Шаг 4. Обучаем дерево решений

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(
    max_depth=3,
    random_state=42
)

model.fit(X_train, y_train)
```

Почему снова дерево решений?

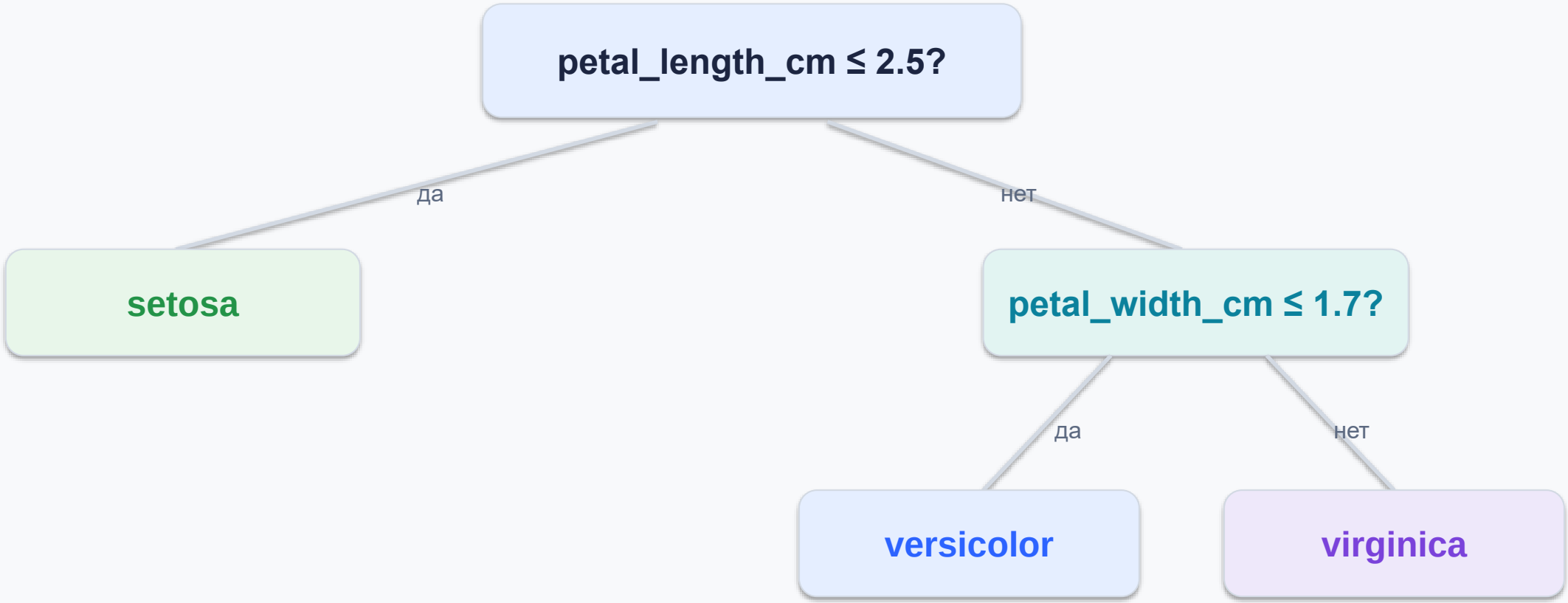
логика похожа на цепочку вопросов; модель подходит для первого практического запуска; меньше новой теории — больше внимания к подготовке данных.

fit()

команда, которая запускает обучение модели на X_train и y_train

Как дерево смотрит на признаки

Схема ниже не обязана совпадать с точным деревом из ноутбука. Она показывает принцип: вопрос по признаку ведёт к следующей ветке.



Шаг 5. Получаем предсказания на тестовых строках

```
predictions = model.predict(X_test)

result = X_test.copy()
result['true_species'] = y_test.values
result['predicted_species'] = predictions

result.head(10)
```

Смотрим несколько строк, а не одну.

Сравниваем правильный класс и ответ модели.

Пока не считаем метрики качества.

Обсуждаем, какие признаки могли повлиять на ответ.

Подробная оценка качества будет на следующем уроке.

Шаг 6. Предсказание для нового объекта

```
new_flower = pd.DataFrame({
    'sepal_length_cm': [5.8],
    'sepal_width_cm': [2.7],
    'petal_length_cm': [4.1],
    'petal_width_cm': [1.0]
})

model.predict(new_flower)
```

Важно

у нового объекта должны быть те же признаки, что были в X при обучении модели

Что обсуждаем

похож ли новый цветок по признакам на одну из групп в таблице

Расширение для СПО: подготовка признаков глубже

Базовый путь

берём числовые признаки и обучаем модель

Если есть текстовые категории

например island или sex в другом датасете

pd.get_dummies()

переводит категории в числовые столбцы

```
X = pd.get_dummies(X)

# модель получает только числа
```

Этот блок можно оставить как расширение: в базовой версии урока все признаки у Iris уже числовые.

Практическая работа: уровни

Уровень 1

загрузить датасет, выделить X и y, обучить дерево решений, получить предсказания

Уровень 2

изменить набор признаков: убрать один столбец или оставить только признаки лепестка; сравнить предсказания по смыслу

Уровень 3

создать несколько новых объектов, получить прогнозы и объяснить, почему ответы выглядят правдоподобно или спорно

Для 10–11 классов можно ограничиться уровнями 1–2. Для СПО уровень 3 лучше давать как основную часть практикума.

Типичные ошибки

label попал в X
модель получила правильный ответ среди признаков

забыли удалить пропуски
fit() может остановиться с ошибкой

новый объект не совпал по столбцам
в predict передали другой набор признаков

не посмотрели df.info()
ошибка была видна ещё до обучения

слишком рано спорят о качестве
сначала надо убедиться, что цепочка работает

Что фиксируем в конце урока

read_csv()
открываем таблицу

X и y
выделяем признаки и класс

dropna()
убираем мешающие пропуски

fit()
обучаем модель

predict()
получаем ответы

Обучающиеся должны уметь пройти путь от реального CSV-файла до первых предсказаний модели.

Следующий шаг — отдельный разбор качества классификации и метрик.